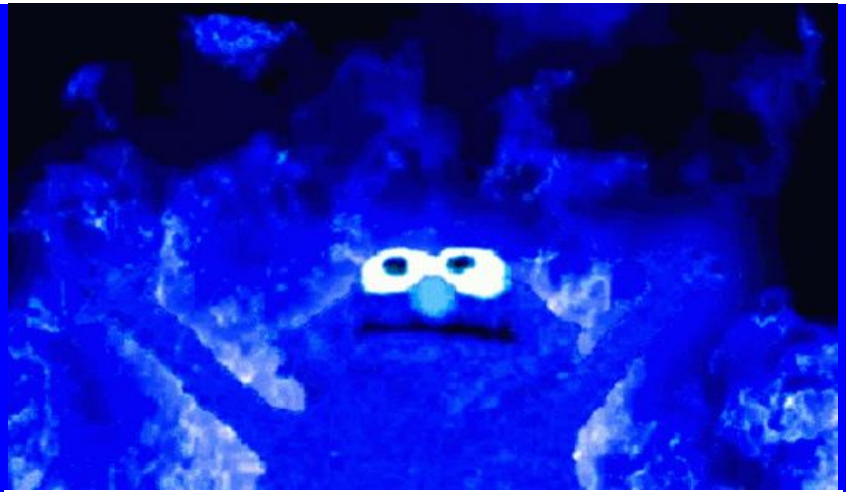
Month 1: Week 1 



Introduction to Cloud Computing

Recap (on what was learned last sem.) & Intros….

Learning Objectives

By the end of this session, you should be able to:

– Define cloud computing and explain its importance in modern software development.

– Compare traditional on-premises infrastructure with cloud infrastructure.

– Differentiate between cloud service models — IaaS, PaaS, and SaaS — and identify real-world examples of each.

– Explain the four major cloud deployment models: Public, Private, Hybrid, and Multi-cloud.

– Describe the Shared Responsibility Model and identify who owns what across example scenarios. Understand the history and evolution of cloud computing.

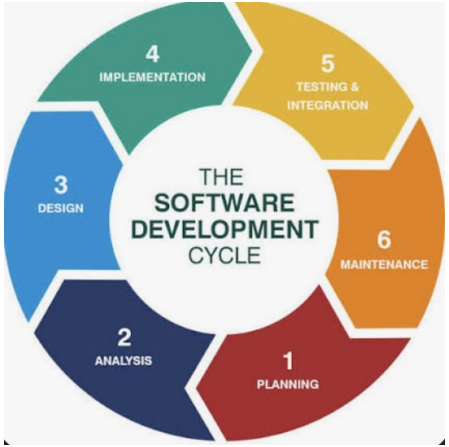
– Situate cloud engineering within the Software Development Lifecycle (SDLC) and explain its role across all phases.

Materials & Resources

1. https://medium.com/@thecloudfairy/before-the-cloud-5ce6986162c2
2. https://aws.amazon.com/compliance/shared-responsibility-model/
3. https://medium.com/@tomcroll/unpacking-the-shared-responsibility-model-for-cloud-security-how-to-avoid-coverage-gaps-and-84b3be463c9b
4. https://youtu.be/RWgW-CgdIk0?si=EQiUkS5pgcv8JpCS
5. https://youtu.be/ZaA0kNm18pE?si=vlNvXGS3US6ZOdcM

The SDLC, System Design

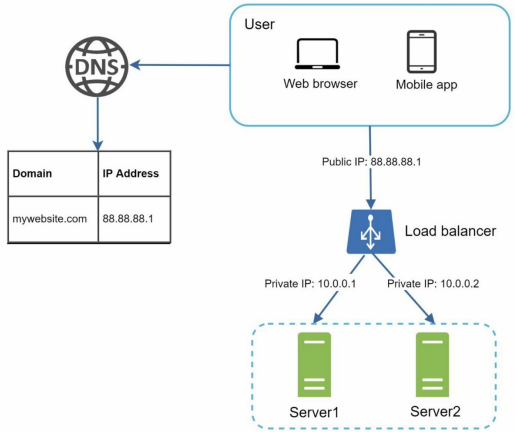
The Software Development Life Cycle (SDLC) is a structured, repeatable process that development teams use to design, build, test, and maintain software. It provides a standardized framework to control costs, minimize risks, and ensure that the final product consistently meets stakeholder and user requirements.



System design is about answering the question: how do we build this so it works for millions of users, doesn't break under pressure, and can be maintained by a real team?

A JavaScript function is one tiny piece of a much larger system. That system needs servers to run on, databases to store data, networks to deliver responses, and monitoring tools to know when something breaks…

Read this: here



What is the Cloud?

The cloud is a global network of remote servers housed in massive data centers. Instead of storing data or running programs on your local hard drive, you access them over the internet.

These data centers are owned by providers, like:

- Amazon Web Services
- Microsoft Azure
- Google Cloud Platform
- DigitalOcean
- Heroku
- Huawei
- Alibaba

Cloud Computing

> Cloud computing is the on-demand delivery of computing resources — servers, storage, databases, networking, software, analytics, and intelligence — over the internet (the “cloud”) to offer faster innovation, flexible resources, and economies of scale.

What is Cloud Engineering?

Cloud engineering involves designing, building, and managing cloud-based systems and services. It supports scalable and flexible IT solutions by leveraging cloud computing resources.

Some Cloud Computing Models:

A. Infrastructure as a Service (IaaS): Provides virtualized computing resources over the internet (e.g., AWS EC2, Google Compute Engine, Azure Virtual machines, DigitalOcean droplets).
B. Platform as a Service (PaaS): Provides hardware and software tools over the internet, typically for application development (e.g., Google App Engine for developing and hosting web apps, Azure App Service).
C. Software as a Service (SaaS): Delivers software applications over the internet (e.g., Google Workspace, Microsoft Office 365).

Importance of Cloud Engineering in the SDLC

The SDLC (Software Development Life Cycle) is the structured process used to build software: Planning → Design → Development → Testing → Deployment → Maintenance.

Cloud engineering impacts every stage of this lifecycle.

- Planning: Architecture & Scalability from Day One
- Development: Environment Consistency
- Testing: On-Demand Infrastructure
- Deployment: CI/CD & Automation
- Maintenance: Monitoring, Scaling, and Reliability

Cloud engineering basically transforms our SDLC from:

“Build → Deploy → Hope it works” into: “Build → Test at scale → Deploy automatically → Monitor → Improve continuously”

Importance of Cloud Engineers in the SDLC

Cloud engineers aren't just people who "turn on servers." In a real team they:

- Collaborate with developers during the **Design** phase to choose the right services and architecture
- Build the infrastructure that lets developers **deploy** code reliably and repeatedly
- Configure security, networking, and access controls
- **Monitor** systems in production and respond to incidents
- Optimize for cost, performance, and reliability at scale

Cloud Deployment Models

Public Cloud: Infrastructure owned and operated by a third-party cloud provider (AWS, Azure, GCP)

Private Cloud: Services are maintained on a private network (e.g., VMware, OpenStack). Infrastructure dedicated to a single organization. Hosted on-premises or by a third party. More control, higher cost. Common in banking and government.

Hybrid Cloud: Combines public and private clouds, allowing data and applications to be shared between them.

Multi-cloud: Using cloud services from more than one provider (e.g. AWS for compute + GCP for AI/ML). Avoids vendor lock-in, increases complexity.



Traditional (on-premises) vs. Cloud Infrastructure comparison

Traditional (On-Premises) Infrastructure

You buy physical servers and store them in your own data center or server room. You pay for everything upfront…hardware, power, cooling, space, and the people to manage it. You provision capacity based on your peak expected load, which means most of the time you're paying for resources you're not using. If you underestimate, you're stuck until you buy more hardware (weeks or months). If something breaks, you fix it yourself.

Cloud Infrastructure

You rent computing resources from a provider like AWS or Azure. You pay only for what you use, and you can scale up or down in minutes. No hardware to buy or maintain. The provider handles the physical infrastructure, redundancy, and data center operations.

Most startups couldn't afford traditional infrastructure. Cloud made it possible to build and scale globally without millions in upfront hardware investment — that's why it transformed the industry.

Sub-Occupations in Cloud Engineering

Cloud Architecture

Role: Designs and manages cloud infrastructure and services, ensuring alignment with business goals and technical requirements. Key Skills: Architectural design, cloud service selection, cost management.

Tools: AWS CloudFormation, Azure Resource Manager, Google Cloud Deployment Manager.

Cloud DevOps Engineer

Role: Implements CI/CD pipelines, automates deployment processes, and maintains cloud infrastructure.

Key Skills: Automation, scripting, continuous integration/deployment.

Tools: Jenkins, Bitbucket Pipelines, GitHub Actions, Terraform, Ansible.

Cloud Security Engineer

Role: Focuses on securing cloud environments, managing access controls, and monitoring for potential security threats.

Key Skills: Security protocols, compliance, risk management.

Tools: AWS Security Hub, Azure Security Center, Google Cloud Security Command Center.

Cloud Sub-Occupations (contd.)

Cloud Network Engineer

Role**:** Designs and manages cloud network architecture, ensuring connectivity and optimizing network performance.

Key Skills: Network design, performance tuning, troubleshooting.

Tools: AWS VPC, Azure Virtual Network, Google Cloud VPC.

Cloud Data Engineer

Role: Manages and optimizes cloud-based data storage, processing, and analytics.

Key Skills: Data warehousing, ETL processes, database management.

Tools: AWS Redshift, Google BigQuery, Azure Synapse Analytics.

Essential Tools and Frameworks | The SEVEN “ANDS” of Cloud Engineering

(1) Infrastructure as Code (IaC)

Purpose: Automates the setup and management of cloud resources using code.

Tools: Terraform, AWS CloudFormation, Azure Resource Manager.

Configuration Management

Purpose: Automates the deployment and configuration of infrastructure.

Tools: Ansible, Chef, Puppet.

(2). Communication & Collaboration: Purpose: To enhance and improve clear communication and teamwork among team members.

Tools: Slack, Jira, WhatsApp, Teams, Discord, Asana, Click-up, Trello, Workplace, Zoom, Google Meet, Telegram, Linear, Notion, Circle.

(3). CI/CD/ & CD: Continuous Integration, Continuous Delivery, and Continuous Deployment

Purpose: Automates the build, test, and deployment of applications.

Tools:** Jenkins, GitHub Actions, GitLab CI/CD, Bitbucket Pipelines, ArgoCD, Vercel, CircleCI, Firebase.



Essential Tools and Frameworks(contd).

(4).Monitoring and Logging

Purpose: Tracks performance, manages logs, and monitors systems for issues.

Tools: AWS CloudWatch, Azure Monitor, Google Cloud Operations Suite, Prometheus, Grafana.

(5). Containerisation & Container Orchestration: Docker, Kubernetes, Docker Swarm, AWS EKS (k8s), ECS (AWS for Docker).

(6). Networking & Security: AWS VPC, Sonarqube, Azure VPN

(7.) AI & Machine Learning: (NEW)

Practice Questions

1. In your own words, define cloud computing. What problem does it solve compared to traditional infrastructure?
2. A startup needs to launch a product in 2 weeks but can't afford servers. What cloud service model would you recommend and why?
3. A bank wants full control over its data and can't use shared infrastructure. Which deployment model suits them best?
4. Who is responsible for patching the operating system in an IaaS setup, the provider or the customer?
5. Name one real-world example each of IaaS, PaaS, and SaaS.